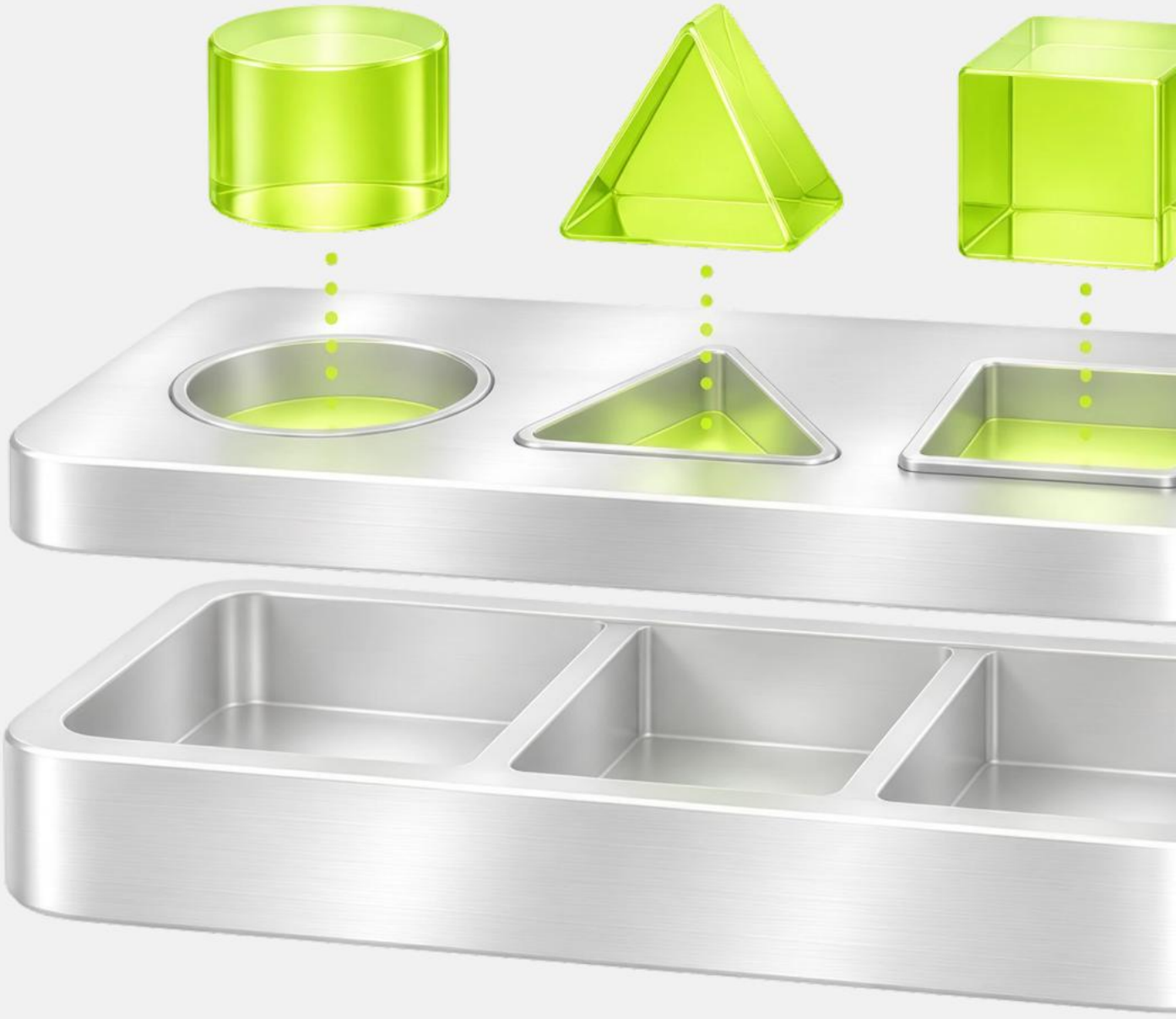
Валидация и тестирование API в Go

БЭКЕНД-СЕРВИС ЗАКАЗОВ



X5 Tech

Стартовый шаблон

В СТАРТОВОМ РЕПОЗИТОРИИ УЖЕ ЕСТЬ

- Go-модуль и базовая структура проекта
- Заготовка in-memory-репозитория
- Модели Order, Item, StatusChange
- Заготовка сервисного слоя
- Интерфейсы основных слоёв
- HTTP-сервер на chi

Стартовый проект компилируется, но оцениваемая бизнес-логика намеренно не реализована

РАБОЧИЙ ЭНДПОИНТ

```
GET /health
```

- Команды запуска и проверки
- Точки расширения
- Заглушки обязательных маршрутов

ЧТО НЕОБХОДИМО РЕАЛИЗОВАТЬ

Доведите заготовку до работающего бэкенд-сервиса заказов

- Используется in-memory-хранилище

НАДЁЖНОСТЬ

- Предметные ошибки и корректные HTTP-коды
- Структурированные логи
- Middleware логирования и восстановления после паники

КАЧЕСТВО

- Swagger/OpenAPI
- Модульные table-driven тесты
- Покрытие сервисного слоя не менее 80%
- Заполненный README

API И БИЗНЕС-ЛОГИКА

- Создание заказа
- Получение одного заказа и списка заказов
- Изменение статуса заказа
- Получение истории статусов
- Валидация входных данных
- Расчёт итоговой стоимости

6 обязательных эндпоинтов

Все обязательные маршруты описаны в Swagger.
Формат запросов и ответов — JSON

Метод	Путь	Назначение	HTTP
GET	/health	Проверка доступности сервиса	200
POST	/api/v1/orders	Создание заказа	201
GET	/api/v1/orders	Получение списка заказов	200
GET	/api/v1/orders/{id}	Получение заказа	200
PATCH	/api/v1/orders/{id}/status	Изменение статуса	200
GET	/api/v1/orders/{id}/history	Получение истории	200

Модель заказа

КАЖДАЯ ПОЗИЦИЯ

```
{
  "name": "Keyboard",
  "quantity": 1,
  "price": 500000
}
```

Цена хранится целым числом в минимальных денежных единицах — например, в копейках.

ЗАКАЗ СОДЕРЖИТ

1. Уникальный идентификатор
2. Непустой список позиций
3. Рассчитанную итоговую стоимость
4. Текущий статус
5. Дату создания
6. Историю изменения статусов

Валидация
и расчёт стоимости

in-memory

ПРИ СОЗДАНИИ ПРОВЕРИТЬ:

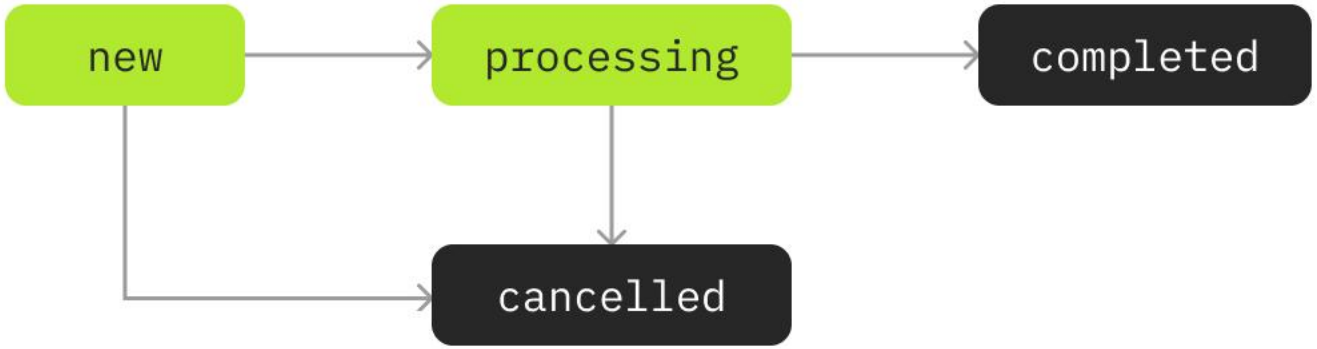
1. Список позиций не пустой
2. Названия товаров заполнены
3. Количество > 0
4. Цена > 0

ИТОГОВАЯ СТОИМОСТЬ

$$total_price = \Sigma\ price \times quantity$$

Жизненный цикл заказа

ДОПУСТИМЫЕ ПЕРЕХОДЫ



новый заказ автоматически получает статус «new»

«completed» и «cancelled» — конечные статусы

недопустимый переход → `409 Conflict`

ДОПУСТИМЫЕ СТАТУСЫ

`new` `processing` `completed` `cancelled`

ИСТОРИЯ СТАТУСОВ

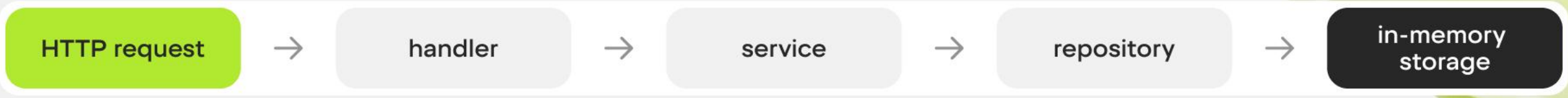
Для каждого изменения статуса добавить запись

1. Новый статус
2. Время изменения

ВСЕ РАЗРЕШЁННЫЕ ПЕРЕХОДЫ

new → processing
new → cancelled
processing → completed
processing → cancelled

Архитектура решения



HANDLER

- Читает HTTP-запрос
- Проверяет формат JSON
- Вызывает сервис
- Формирует HTTP-ответ
- Преобразует ошибки в HTTP-коды

SERVICE

- Содержит бизнес-логику
- Выполняет валидацию
- Рассчитывает стоимость
- Проверяет переходы статусов
- Формирует историю

REPOSITORY

- Сохраняет заказы
- Возвращает заказ по идентификатору
- Возвращает список заказов

Зависимости передаются через конструкторы. Репозиторий описывается интерфейсом

Структуру проекта можно изменить, если зоны ответственности остаются разделены

Единый контракт ошибок

ЕДИНЫЙ ФОРМАТ ОШИБКИ

```
{
  "error": "status transition is not allowed",
  "code": "INVALID_STATUS_TRANSITION"
}
```

Внутренние детали клиенту не возвращаются

Проверка: errors.Is и при необходимости errors.As

ОБЯЗАТЕЛЬНЫЕ HTTP-КОДЫ

400 Bad Request
Некорректный JSON / невалидные данные

404 Not Found
Заказ не найден

409 Conflict
Недопустимый переход статуса

500 Internal Error
Неожиданная внутренняя ошибка

Логирование, recovery и Swagger

Наблюдаемость каждого запроса и актуальная документация API

КАЖДЫЙ ЗАПРОС ЛОГИРУЕТСЯ

`method` `path` `status` `duration` `request_id`

RECOVERY MIDDLEWARE

Перехватывает панику, записывает её в лог
и возвращает безопасный ответ 500

SWAGGER / OPENAPI

Все обязательные маршруты

Параметры и тела запросов

Успешные ответы

Ошибки и HTTP-коды

Тестирование: обязательные сценарии

СОЗДАНИЕ, ВАЛИДАЦИЯ И ЧТЕНИЕ

01 Успешное создание заказа

02 Создание заказа без позиций

03 Пустое название товара

04 Нулевые и отрицательные quantity и price

05 Расчёт итоговой стоимости

06 Получение существующего и отсутствующего заказа

СПИСОК, СТАТУСЫ И ИСТОРИЯ

01 Получение списка заказов

02 Допустимые переходы статусов

03 Недопустимые переходы

04 Запрет изменения конечного статуса

05 Добавление записей в историю

06 Хронологический порядок истории

Тестирование: инструменты и покрытие

≥80%

Покрытие пакета service

```
go test ./...
```

ТРЕБОВАНИЯ К ТЕСТАМ

- Table-driven tests
- Стандартный пакет testing
- testify/assert или testify/require
- Подмена репозитория через интерфейс

Интеграционные и HTTP API-тесты на текущем контроле не обязательны

Критерии оценивания

КРИТИЧЕСКИЕ УСЛОВИЯ

1. Проект компилируется и запускается
2. Присутствует отдельный слой бизнес-логики
3. Написаны модульные тесты
4. Решение не размещено целиком в main.go
5. Работа находится в Git-репозитории
6. Работа выполнена самостоятельно

РАСПРЕДЕЛЕНИЕ БАЛЛОВ

Функциональная полнота	16
Модульное тестирование	10
Документация API	5
Качество кода и организация проекта	9
Максимальное количество баллов	**40**

Формат сдачи

Работа выполняется индивидуально. Сдавайте только проверенную версию из указанной ветки репозитория.

СОСТАВ РЕПОЗИТОРИЯ

- Ссылка на git-репозиторий
- Название ветки с решением
- Исходный код
- go.mod, go.sum
- Swagger-документация
- .gitignore
- заполненный README

КОМАНДЫ ПРОВЕРКИ

```
go test ./...

go test ./internal/service/... -cover

go vet ./...

go build ./...
```



Все команды должны завершаться успешно

Финальный чек-лист

Сдавайте только проверенную версию
из указанной ветки репозитория.



- [ ] Шесть эндпоинтов
- [ ] Единый JSON-формат ошибок
- [ ] Переходы статусов
- [ ] Покрытие service от 80%
- [ ] Доступен Swagger UI
- [ ] Нет секретов и артефактов сборки
- [ ] README соответствует реализации